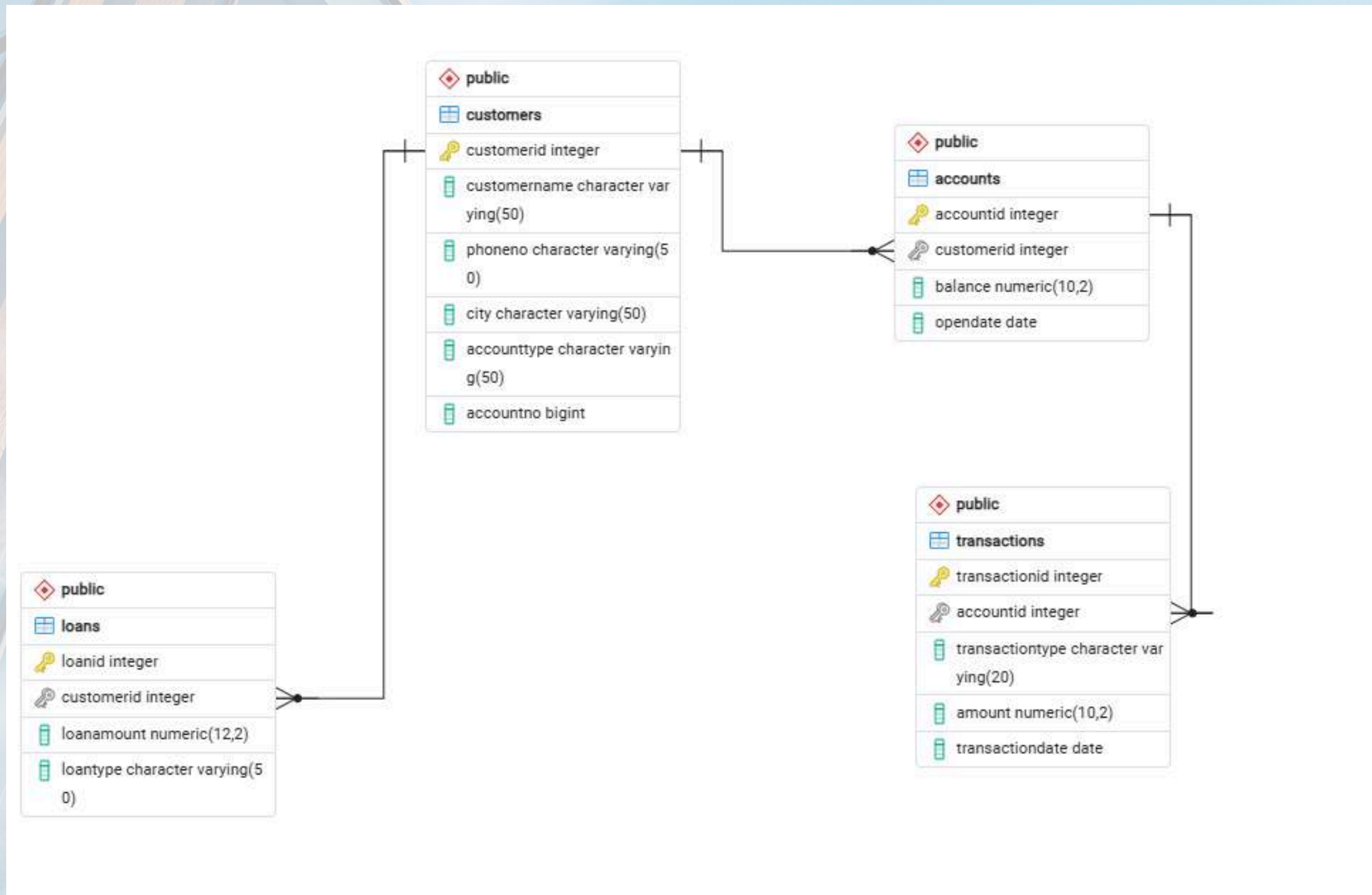


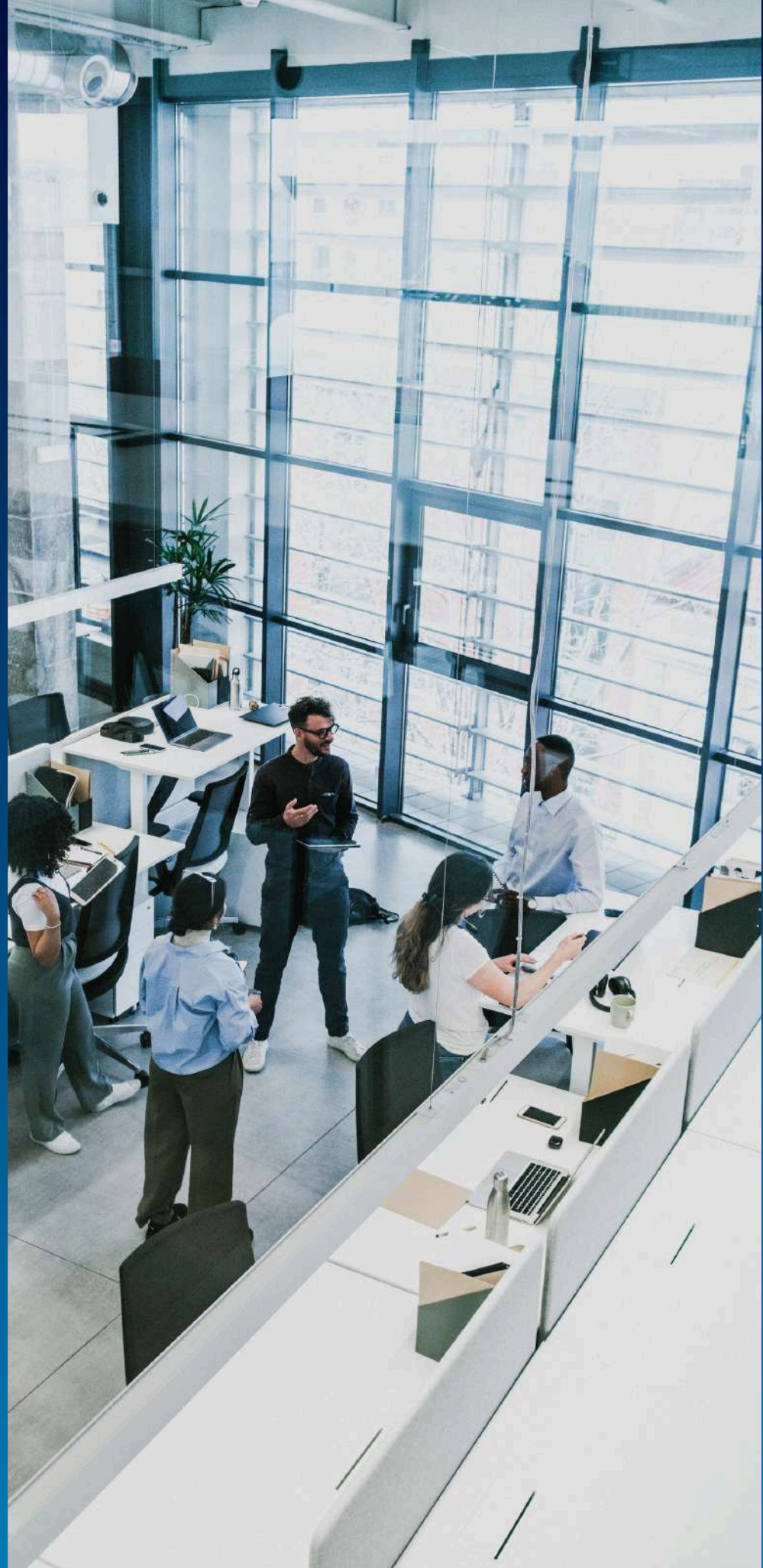
Manage customer
records

BANK MANAGEMENT SYSTEM

PRESENTED BY
SAKSHI ANKATWAR

Entity Relationship Diagram





1. Display customer names, account numbers, and account balances using INNER JOIN.

```
select c.customerid,  
       c.customername ,  
       c.accountno,  
       a.balance  
from customers c  
inner join accounts a  
on c .customerid=a.customerid;
```

customerid integer	customername character varying (50)	accountno bigint	balance numeric (10,2)
1	Rahul Sharma	1001	55000.00
2	Sneha Patil	1002	120000.00
3	Aman Verma	1003	35000.00
4	Priya Singh	1004	98000.00
5	Karan Mehta	1005	75000.00
6	Neha Joshi	1006	150000.00
7	Rohit Kumar	1007	42000.00

2. Find the top 3 customers with the highest account balances

```
select c.customerid,  
       c.customername ,  
       a.balance  
from customers c  
inner join accounts a  
on c .customerid=a.customerid  
order by a.balance desc  
limit 3;
```

customerid 	customername 	balance 
integer	character varying (50)	numeric (10,2)
9	Vivek Shah	200000.00
6	Neha Joshi	150000.00
2	Sneha Patil	120000.00



3. Show all customers who have taken loans along with loan amount and loan type

```
select c.customername ,  
       l.loanamount,  
       l.loantype  
from customers c  
inner join loans l  
on c .customerid=l.customerid ;
```

customername character varying (50) 🔒	loanamount numeric (12,2) 🔒	loantype character varying (50) 🔒
Rahul Sharma	500000.00	Home Loan
Sneha Patil	200000.00	Car Loan
Priya Singh	100000.00	Education Loan
Karan Mehta	300000.00	Business Loan
Neha Joshi	150000.00	Personal Loan
Pooja Sharma	250000.00	Home Loan
Vivek Shah	400000.00	Business Loan
Anjali Verma	180000.00	Car Loan

4. Find the total deposited amount and total withdrawn amount separately

```
select
    transactiontype,
    SUM(amount) as total_amount
from transactions
where transactiontype in ('Deposit', 'Withdraw')
group by transactiontype;
```



transactiontype character varying (20) 🔒	total_amount numeric 🔒
Deposit	121000.00
Withdraw	35000.00

5. Display customer-wise total transaction amount using GROUP BY.

```
SELECT c.customerid,c.customername,
       SUM(t.amount) AS total_transactionamount
FROM customers c inner join accounts a
on c.customerid =a.customerid
inner join transactions t
on a.accountid =t.accountid
GROUP BY c.customerid ,c.customername;
```



customerid [PK] integer	customername character varying (50)	total_transactionamount numeric
9	Vivek Shah	30000.00
3	Aman Verma	8000.00
5	Karan Mehta	15500.00
4	Priya Singh	15000.00
10	Anjali Verma	2000.00
6	Neha Joshi	7000.00
2	Sneha Patil	35000.00

6. Find customers whose balances are greater than the average bank balance.

```
select  c.customername,  
        c.customerid,  
        a.balance  
  from  customers c  
  inner join accounts a  
on c.customerid=a.customerid  
where a.balance > ( select avg(balance)  
                   FROM accounts );
```

customername character varying (50) 🔒	customerid integer 🔒	balance numeric (10,2) 🔒
Sneha Patil	2	120000.00
Priya Singh	4	98000.00
Neha Joshi	6	150000.00
Vivek Shah	9	200000.00

7. Show the highest transaction amount performed by each customer

```
select c.customerid,  
       c.customername,  
       max(t.amount) as highest_amount from customers c  
inner join accounts a  
  on c.customerid = a.customerid  
inner join transactions t  
  on a.accountid = t.accountid  
group by c.customerid, c.customername;
```

customerid [PK] integer	customername character varying (50)	highest_amount numeric
9	Vivek Shah	30000.00
3	Aman Verma	5000.00
5	Karan Mehta	12000.00
4	Priya Singh	15000.00
10	Anjali Verma	2000.00
6	Neha Joshi	7000.00
2	Sneha Patil	25000.00
7	Rohit Kumar	15000.00
1	Rahul Sharma	10000.00
8	Pooja Sharma	4500.00



8. Display all customers who have not taken any loans using LEFT JOIN

```
select  c.customerid,  
        c.customername  
from    customers c  
left join loans l  
    on  c.customerid = l.customerid  
where  l.customerid is null;
```

customerid [PK] integer	customername character varying (50)
3	Aman Verma
7	Rohit Kumar

9. Find the total number of transactions performed by each customer.

```
select c.customerid,  
       c.customername,  
count(t.transactionid) as total_transaction  
from   customers c  
inner join accounts a  
on c.customerid =a.customerid  
inner join transactions t  
on a.accountid =t.accountid  
group by c.customerid, c. customername;
```

customerid [PK] integer	customername character varying (50)	total_transaction bigint
9	Vivek Shah	1
3	Aman Verma	2
5	Karan Mehta	2
4	Priya Singh	1
10	Anjali Verma	1
6	Neha Joshi	1
2	Sneha Patil	2
7	Rohit Kumar	2
1	Rahul Sharma	2

10. Rank customers based on their account balances using **RANK()** window function

```
select c.customerid,  
       c.customername,  
       a.balance,  
  
rank () over(order by a.balance asc)  
as balance_rank  
from customers c  
inner join accounts a  
on c.customerid=a.customerid;
```

customerid integer	customername character varying (50)	balance numeric (10,2)	balance_rank bigint
3	Aman Verma	35000.00	1
7	Rohit Kumar	42000.00	2
1	Rahul Sharma	55000.00	3
10	Anjali Verma	67000.00	4
5	Karan Mehta	75000.00	5
8	Pooja Sharma	88000.00	6
4	Priya Singh	98000.00	7
2	Sneha Patil	120000.00	8
6	Neha Joshi	150000.00	9
9	Vivek Shah	200000.00	10

11. Display dense ranking of customers according to balance using DENSE_RANK()

```
select c.customerid,  
       c.customername,  
       a.balance,  
       dense_rank () over(order by a.balance asc)  
as rank_balance  
from customers c inner join accounts a  
on c.customerid=a.customerid;
```

customerid integer	customername character varying (50)	balance numeric (10,2)	rank_balance bigint
3	Aman Verma	35000.00	1
7	Rohit Kumar	42000.00	2
1	Rahul Sharma	55000.00	3
10	Anjali Verma	67000.00	4
5	Karan Mehta	75000.00	5
8	Pooja Sharma	88000.00	6
4	Priya Singh	98000.00	7
2	Sneha Patil	120000.00	8

12. Show previous transaction amount using LAG() function

```
select transactionid,  
       accountid,  
       amount,  
       lag(amount) over ( partition by accountid  
                          order by transactionid  
                          ) as previous_amount  
from transactions;
```

transactionid [PK] integer	accountid integer	amount numeric (10,2)	previous_amount numeric
1	1001	10000.00	[null]
2	1001	5000.00	10000.00
3	1002	25000.00	[null]
12	1002	10000.00	25000.00
4	1003	3000.00	[null]
13	1003	5000.00	3000.00
5	1004	15000.00	[null]

13. Show next transaction amount using LEAD() function

```
select transactionid,  
       accountid,  
       amount,  
       lead(amount) over (  
           partition by accountid  
           order by transactionid  
       ) as upcoming_amount  
from transactions;
```

transactionid [PK] integer	accountid integer	amount numeric (10,2)	upcoming_amount numeric
1	1001	10000.00	5000.00
2	1001	5000.00	[null]
3	1002	25000.00	10000.00
12	1002	10000.00	[null]
4	1003	3000.00	5000.00
13	1003	5000.00	[null]
5	1004	15000.00	[null]
6	1005	12000.00	3500.00
14	1005	3500.00	[null]

14. Calculate running total of transaction amounts using SUM() OVER().

```
select transactionid,  
       amount,  
       sum(amount) over (  
         order by transactionid  
       ) as running_total  
from transactions;
```

transactionid [PK] integer	amount numeric (10,2)	running_total numeric
1	10000.00	10000.00
2	5000.00	15000.00
3	25000.00	40000.00
4	3000.00	43000.00
5	15000.00	58000.00
6	12000.00	70000.00
7	7000.00	77000.00
8	9000.00	86000.00
9	4500.00	90500.00

15. Find the second highest account balance using subquery or window function.

```
select balance
from ( select balance,dense_rank() over (
      order by balance desc) as rank_no
      from accounts)
where rank_no = 2;
```

balance	🔒
numeric (10,2)	
150000.00	

16. Display customer-wise minimum and maximum transaction amounts.

```
SELECT c.customername,  
       MIN(t.amount) AS minimum_transaction,  
       MAX(t.amount) AS maximum_transaction  
FROM customers c  
inner JOIN accounts a  
    ON c.customerid = a.customerid  
inner JOIN transactions t  
    ON a.accountid = t.accountid  
GROUP BY c.customerid, c.customername;
```

customername character varying (50) 🔒	minimum_transaction numeric 🔒	maximum_transaction numeric 🔒
Vivek Shah	30000.00	30000.00
Aman Verma	3000.00	5000.00
Karan Mehta	3500.00	12000.00
Priya Singh	15000.00	15000.00
Anjali Verma	2000.00	2000.00
Neha Joshi	7000.00	7000.00
Sneha Patil	10000.00	25000.00
Rohit Kumar	9000.00	15000.00

THANK YOU

Thank you for taking the time to explore this presentation. Your attention and interest are highly valued.